

PMPSI: Privacy-Preserving Predicate-Supporting Multi-Party Private Set Intersection

Chengzhi Gao^{ID}, Yihan Bao^{ID}, Chang Xu^{ID}, Liehuang Zhu^{ID}, *Senior Member, IEEE*, Hongyi Liu, Kashif Sharif^{ID}, *Senior Member, IEEE*, and Pengbo Wang^{ID}

Abstract—Multi-Party Private Set Intersection (MPSI) protocol allows multiple participants to secretly compute their data intersection without any additional disclosure. However, conventional MPSI protocols treat all participants as identical entities and fail to accommodate the diverse characteristics of participants, which may lead to intersection results that do not align with the requestor's intent. To this end, we introduce a new cryptographic primitive of PMPSI, wherein only participants whose attributes satisfy a predicate defined by a designated requestor are eligible to contribute to the intersection computation, termed Predicate-Supporting Multi-party Private Set Intersection (PMPSI). PMPSI enables conditional intersection with strong privacy guarantees. Specifically, PMPSI ensures that the predicate, participant attributes, and match status remain confidential to all entities involved. To instantiate this primitive under the honest-but-curious adversarial model, we design a dual-mode oblivious comparison (DMOC) protocol with two operational versions as a core subroutine and integrate it with threshold homomorphic encryption and encrypted inverse Bloom filter. We formally prove the security of the protocol via simulation-based arguments and implement a prototype to evaluate its efficiency. Experimental results demonstrate that the PMPSI protocol achieves linear computational and communication complexity with respect to the number of participants, confirming its practicality for realistic application scenarios such as secure cross-organizational collaboration and privacy-preserving resource sharing in mobile networks.

Index Terms—Multi-party private set intersection, privacy preserving, predicate.

I. INTRODUCTION

WITH the explosive growth of data and the rapid evolution of digital technology, user demands for data processing have soared, requiring not only efficiency but also seamless collaboration and robust privacy preservation. Traditional cloud computing, while capable of facilitating collaborative data processing and offloading heavy computational burdens from users

to platforms, may pose privacy risks such as the leakage of sensitive data and processing methodologies [1], [2]. Fortunately, the advent of secure multi-party computation (MPC) offers a promising solution to these challenges. MPC allows that sensitive data can be processed and analyzed collaboratively without revealing the raw data to any party.

Private set intersection (PSI), a representative application of MPC, allows two parties to securely compute the intersection of their private sets without revealing any additional information [3], [4], [5]. Multi-party PSI (MPSI) is the natural extension of PSI to scenarios involving more than two parties. During the past several years, MPSI has been applied to a variety of practical domains, including secure sharing of sensitive security incident information [6], measuring the effectiveness of online advertising [7], vertical federated learning [8], and identifying high-risk individuals during disease outbreaks [9].

However, in practical application scenarios, user groups often possess diverse attribute characteristics. Individual users may carry multiple identity tags, including industry attributes (e.g., healthcare, finance, education), device types (smart wearables, industrial sensors), and security levels (regular users, VIP users). Such complex attribute compositions expose fundamental limitations of conventional MPSI techniques. Specifically, their undifferentiated processing mechanisms, which treat all participants as homogeneous entities, fail to meet the fine-grained control requirements for privacy-preserving data computation across varied scenarios.

Therefore, in this work, we consider a more adaptive and practical MPSI setting in which participation in the set intersection is governed by a predicate. As illustrated in Fig. 1, a special initiating party, referred to as the requestor, seeks to compute the intersection only with parties whose attributes satisfy a specified predicate. In other words, the requestor defines a predicate f to identify a user P_i who satisfies $f(\vec{x}_i) = 1$, where x_i denotes the attribute vector of P_i , and performs the intersection computation exclusively with those users.

A naive implementation of this functionality could integrate basic attribute-based encryption (ABE) or predicate encryption (PE) with an MPSI protocol. Specifically, it would require an external trusted server to first execute ABE or PE to identify users who satisfy the predicate, and then proceed with the MPSI protocol among those selected users. However, in practical deployment scenarios, stringent privacy requirements apply. The attributes of each party, their dataset, and even the predicate defined by the requestor may be sensitive. In such cases, the

Received 20 January 2026; revised 14 May 2026; accepted 11 June 2026. Date of publication 17 June 2026; date of current version 1 September 2026. This work was supported in part by the National Key Research and Development Program of China under Grant 2024YFB2705300 and in part by the National Natural Science Foundation of China under Grant 61972037. (Corresponding author: Chang Xu.)

Chengzhi Gao, Yihan Bao, Chang Xu, Liehuang Zhu, Hongyi Liu, and Pengbo Wang are with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: chengzhigao@bit.edu.cn; baoyihan09@gmail.com; xuchang@bit.edu.cn; liehuangz@bit.edu.cn; liu.hongyi96@foxmail.com; wangpengbo@bit.edu.cn).

Kashif Sharif is with the School of Computer Science and Technology, Beijing Institute of Technology, Beijing 100811, China (e-mail: 7620160009@bit.edu.cn).

Digital Object Identifier 10.1109/TDSC.2026.3704390

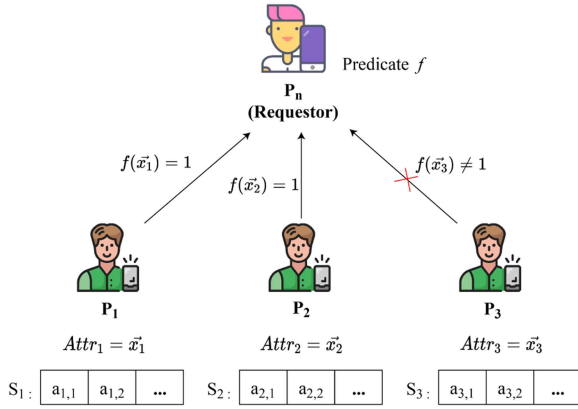


Fig. 1. Example of the MPSI protocol governed by predicate.

requestor prefers to keep the predicate private, and each client is unwilling to disclose its attributes or reveal whether it satisfies the predicate. Clearly, a protocol constructed by simply combining ABE or PE with MPSI fails to satisfy these privacy requirements, as sensitive information and match status could be exposed to the external server. This gives rise to a critical research challenge: *how to design a predicate-constrained MPSI protocol that enables intersection computation only among qualifying parties, while simultaneously providing provable privacy guarantees for attributes, predicate, and match status.*

To demonstrate the practicality and flexibility of our framework, we present two representative application scenarios: traditional cross-organizational collaboration among stable server-side entities, and privacy-preserving resource sharing in mobile network environments, where nearby devices collaboratively coordinate bandwidth resources under dynamic participation conditions while protecting sensitive device information.

Advertisement Recommendation: In cross-platform ad recommendation, platforms can improve targeting by selectively collaborating with partners. MPSI enables privacy-preserving data sharing, while predicates allow the platform to control which partners participate, improving both accuracy and efficiency. Since participants typically operate on reliable server-side infrastructure, they can support the required cryptographic operations. Keeping the selection criteria confidential preserves competitive advantage and prevents partners from manipulating their data or strategies, thereby ensuring reliable and accurate recommendations.

Resource Sharing in Mobile Networks: In collaborative mobile networking, nearby devices can share local bandwidth resources to improve regional connectivity. MPSI enables privacy-preserving coordination, while predicates allow only devices meeting conditions such as sufficient battery level to participate in bandwidth scheduling. The coordinator learns only the aggregated available bandwidth and time intervals, without identifying specific devices, accessing complete bandwidth profiles, or revealing which devices satisfy the predicate. Devices excluded by the predicate also leak no information about their status or availability.

A. Our Contributions

In this work, we consider a more practical and adaptive conditional MPSI scenario to meet the demands for fine-grained data sharing and privacy preservation. In this framework, a special entity known as the requestor defines a predicate and seeks to compute the set intersection of elements from all parties that satisfy the predicate, while excluding those that do not. Building upon this framework, we propose a novel cryptographic primitive called Predicate-Supporting Multi-Party Private Set Intersection (PMPSI), which enables private set intersection exclusively among users satisfying the predicate. Furthermore, PMPSI achieves enhanced privacy preservation through fourfold confidentiality guarantees: (1) the predicate remains hidden from participants, (2) participant attributes remain hidden from the requestor, (3) elements not in the intersection remain confidential to all parties, and (4) the match status of each participant is concealed from all entities. Notably, the entire intersection computation is conducted under full confidentiality.

In addition, we provide a concrete realization of this primitive under the honest-but-curious adversarial model, referred to as the PMPSI protocol. At the core of PMPSI lies a novel building block, termed dual-mode oblivious comparison (DMOC), which enables parties to jointly evaluate predicate satisfaction without revealing sensitive attribute information or match status. To accommodate different deployment assumptions, we design two variants of DMOC: a basic version that relies on a semi-honest computing service provider (CSP), and a CSP-free version that eliminates this assumption while preserving the same security guarantees. By combining DMOC with threshold homomorphic encryption and encrypted inverse Bloom filters, the protocol achieves comprehensive privacy guarantees across four dimensions: predicate hiding, attribute hiding, match-status hiding, and element hiding. We formally prove the security of PMPSI using a simulation-based argument. Finally, we validate the protocol's efficiency through a series of empirical experiments.

Our contributions can be summarized as follows:

- We introduce a new cryptographic primitive, Predicate-Supporting Multi-party Private Set Intersection, and provide a formal definition that captures both its functionality and security. PMPSI enables intersection computation among parties satisfying a specified predicate while ensuring enhanced privacy guarantees, including predicate hiding, attribute hiding, and match status hiding.
- We present a concrete construction of the PMPSI protocol by integrating three key cryptographic components: encrypted inverse Bloom filters, threshold homomorphic encryption, and our novel dual-mode oblivious comparison sub-protocol.
- We formally prove that the PMPSI protocol satisfies the defined privacy guarantees under the simulation-based security model. In addition, we conduct extensive experiments to evaluate its efficiency.

B. Related Works

Research on classical two-party PSI has reached a relatively mature stage, primarily encompassing two design approaches:

one based on oblivious transfer extension (OTE) [13], and the other based on homomorphic encryption (HE). OTE is a “batch” extension of oblivious transfer, which enables the realization of an (l, m) OT from an (m, k) OT with only a small additional cost [13]. Although OTE can be directly used to construct PSI protocols, it is more commonly adopted as a foundational component in higher-level cryptographic primitives, such as Oblivious Pseudo-Random Functions (OPRF) [14], Oblivious Programmable Pseudo-Random Functions (OPPRF) [15], [16], and Oblivious Key-Value Stores (OKVS) [3], [17], which in turn support the development of more efficient and secure PSI protocols. The work in [18] achieves state-of-the-art performance and security against malicious adversaries by employing OKVS and subfield vector oblivious linear evaluation.

In comparison, HE-based PSI protocols typically incur lower communication costs but higher computational overheads than OT-based PSI [19]. For example, the scheme in [20] uses additive homomorphic encryption to encrypt Bloom filters and identifies intersection elements by comparing the relevant encrypted bits. Fully Homomorphic Encryption (FHE) has also been used in PSI protocols; for instance, in [21], elements of the sets are directly encrypted, and intersections are identified by evaluating whether the product of ciphertexts yields zero under homomorphic multiplication.

In the multi-party setting, PSI can be extended from the two-party case either directly or through recursive constructions. The protocol proposed in [9] extends the scheme from [20] to multiple parties by implicitly determining the common intersection through individual pairwise intersections. Recursive PSI schemes [22] reduce an n -party PSI to a sequence of $(n-1)$ -party PSI executions, eventually simplifying to a sequence of two-party PSI operations. Recently, Zhao et al. [23] proposed a multi-party PSI protocol with one-round online interaction, significantly reducing the online latency and improving practical efficiency. Another widely used technique for MPSI is zero-sharing [10], [11], wherein each party shares values such that their sum equals zero if and only if a specific item appears in all sets.

Beyond the basic functionality of computing set intersections, recent studies have explored advanced PSI variants, such as size-hiding PSI and circuit PSI. Size-hiding PSI [24] ensures that the protocol does not leak the size of the server’s dataset during the intersection process. Works in [25] and [12] propose two-party and multi-party size-hiding PSI protocols, respectively. In contrast, circuit PSI generalizes the PSI output to the evaluation of an arbitrary function over the intersection, denoted as $f(s \cap c)$, rather than revealing the intersection itself. For example, the work in [26] constructs a batch OPRF using VOLE to support secret sharing of the intersection result, effectively realizing a special case of circuit PSI.

The idea of incorporating predicates into PSI was first introduced in [27]. The proposed PSI protocol does not aim to compute a standard intersection between a client and a server, but instead returns only those elements that appear in the intersection and additionally satisfy a specific predicate. Specifically, each element s_i held by the server S is associated with an attribute x_i . The client defines a predicate P and obtains the intersection

$\{c_j \mid \exists s_i \text{ s.t. } c_j = s_i \wedge P(x_i) = 1\}$. A subsequent enhancement in [28] allows the server to retrieve the full intersection independently of the predicate evaluation.

It is important to emphasize that our definition of Predicate-Supporting multi-party PSI differs significantly from these prior works. In our setting, the predicate applies to the attributes of the parties themselves, not to those of the individual elements. Consequently, our protocol must implicitly identify the parties that satisfy the predicate and then compute the intersection only with those parties, without revealing their identities or participation status.

In summary, to provide a clear overview and highlight the advantages of our proposed PMPSI protocols, Table I summarizes representative MPSI schemes in terms of computation, communication, security, and privacy features.

II. PRELIMINARIES

In this section, we first introduce the notations used throughout the paper. Subsequently, we present several fundamental prerequisites for our work: Bloom Filter, threshold partially homomorphic encryption, and threshold fully homomorphic encryption.

A. Notation

This section summarizes the key notations utilized throughout this paper, as presented in Table II. We denote by $\mathbb{Z}_n$ the residue class ring modulo n , and by $\mathbb{Z}_q$ a finite field of prime order q . All participating parties and algorithms are assumed to be probabilistic polynomial-time (PPT) entities. All cryptographic operations, homomorphic evaluations, and data structures adhere to the notation defined in Table II.

B. Bloom Filter

The Bloom Filter [29] is a probabilistic data structure for efficient set membership queries. It allows false positives but guarantees no false negatives.

To construct an m -bit Bloom Filter for a set S :

- 1) Select k hash functions $h_1, \dots, h_k$ mapping $0, 1^*$ to $[m]$.
- 2) Initialize an m -bit array with all bits set to 0.
- 3) For each $x \in S$, compute $h_1(x), \dots, h_k(x)$ and set the corresponding bit positions to 1.

Membership query is performed by checking whether all k hashed positions of an element are set to 1.

By flipping each bit of a Bloom Filter, we obtain an inverse Bloom Filter (**IBF**). Encrypting each bit of the **IBF** yields the encrypted inverse Bloom Filter (**EIBF**). Determining whether an element belongs to the **EIBF** follows a process similar to that of the standard Bloom Filter, involving verification that the index positions corresponding to the hash outputs are equal to $Enc(0)$.

C. Threshold Partially Homomorphic Encryption

We adopt threshold partially homomorphic encryption (TPHE) to protect client attributes. The secret key is split into two shares held by different parties, requiring joint decryption.

TABLE I

COMPARISON OF REPRESENTATIVE MPSI PROTOCOLS. n DENOTES THE NUMBER OF PARTICIPATING PARTIES. m DENOTES THE AVERAGE SET SIZE PER PARTY. λ DENOTES THE STATISTICAL SECURITY PARAMETER, WHICH DETERMINES THE FALSE-POSITIVE RATE OF THE BLOOM FILTERS; AND κ DENOTES THE COMPUTATIONAL SECURITY PARAMETER. $\log|X|$ DENOTES THE CIPHERTEXT X IN BITS. a DENOTES THE LENGTH OF A CLIENT'S ATTRIBUTE VECTOR. t DENOTES THRESHOLD OF HOMOMORPHIC PKE.

Protocol	Computation	Communication	Security	Predicate Support	Privacy Features
Bay's Threshold MPSI [9]	$O(mnt\log X)$	$O(mn)$	Semi-honest	✗	Element hiding only
Wei's Zero-sharing MPSI [10]	$O(nm\lambda)$	$O(m(\log m)^2)$	Semi-honest	✗	Element hiding only
Gao's Bicentric Zero-sharing MPSI [11]	$O((n+m)\lambda)$	$O(nm\lambda)$	Semi-honest	✗	Element hiding only
Shinagawa's Size-hiding MPSI [12]	$O(nm\lambda)$	$O(nm\lambda)$	Semi-honest	✗	Element + Size hiding
ours PMPSI with DMOC	$O(\lambda nm + an)$	$O(nm + n)$	Semi-honest	✓	Four dimensions hiding
ours PMPSI with bDMOC-DC	$O(\lambda nm + an)$	$O(nm + n^2)$	Semi-honest	✓	Four dimensions hiding

TABLE II
SUMMARY OF NOTATIONS

Notation	Description
$P_i, i \in [1, n]$	Participants, where P_n is the requestor
TA	Trusted authority
$Enc_{FHE}(\cdot)$	Threshold FHE scheme
$Enc_{PHE}(\cdot)$	Threshold PHE scheme
$+_c, \sum$	Ciphertext homomorphic addition
$\times_c$	Ciphertext homomorphic multiplication
$Attr_i = \{x_1^i, \dots, x_t^i\}$	Attribute of P_i
$f = \vec{v} = \{v_1, \dots, v_t\}$	Predicate held by requestor
$EAttr_i$	Encrypted attributes of P_i
$S_i = \{a_1^i, \dots, a_m^i\}$	Dataset of P_i
BF	Bloom Filter
$h_1(\cdot), \dots, h_k(\cdot)$	k independent hash functions used in the construction of BF
IBF	Inverse Bloom Filter
EIBF	Encrypted inverse Bloom Filter
π_i	Permutation sequence held by client i
$fvalue_i$	Result of predicate computation (plaintext)
$Fvalue_i$	Transformed predicate calculation results
$Fcor, fcor, cor_i$	Variables used in error correction mechanisms
$Value_j'$	Encrypted indicator denoting whether element j belongs to the intersection

We instantiate TPHE using the Paillier cryptosystem [30], [31], defined as follows:

- 1) $(pk, sk_1, sk_2) \leftarrow \text{KeyGen}(1^\lambda)$: Generate primes $p = 2p' + 1$, $q = 2q' + 1$, set $N = pq$, and $h = \text{lcm}(p-1, q-1)$. Split h into sk_1, sk_2 such that $sk_1 + sk_2 \equiv 0 \pmod{h}$ and $sk_1 + sk_2 \equiv 1 \pmod{N}$.
- 2) $c \leftarrow \text{Enc}(m, pk)$: For $m \in \mathbb{Z}_N$, output $c = (1 + mN) \cdot r^N \pmod{N^2}$, where $r \leftarrow \mathbb{Z}_N^*$.
- 3) $\hat{m}_b \leftarrow \text{PDec}(c, sk_b)$: Output $\hat{m}_b = c^{sk_b} \pmod{N^2}$, for $b \in \{0, 1\}$.
- 4) $m \leftarrow \text{TDec}(\hat{m}_1, \hat{m}_2)$: Output $m = L(\hat{m}_1 \cdot \hat{m}_2 \pmod{N^2}) \pmod{N}$, where $L(x) = (x - 1)/N$.
- 5) $c' \leftarrow \text{Csum}(c_1, c_2)$: Output $c' = c_1 \cdot c_2$, encrypting $m_1 + m_2$.
- 6) $c' \leftarrow \text{Smul}(c, k)$: Output $c' = c^k$, encrypting $k \cdot m$.

D. Threshold Fully Homomorphic Encryption and Rotation

We employ threshold fully homomorphic encryption (FHE) to protect the information encoded in Bloom filters. TFHE guarantees that ciphertexts can only be decrypted to the original plaintext when a threshold number of participants jointly perform the decryption. Moreover, modern lattice-based FHE schemes support homomorphic rotation, which permutes packed

plaintext slots within a ciphertext to enable efficient SIMD-style vectorized computation. Due to the complexity of lattice-based constructions, we omit the underlying details and focus on the functionality; interested readers are referred to [32]. The scheme consists of the following algorithms:

- 1) $(pk, sk_1, \dots, sk_n) \leftarrow \text{KeyGen}(1^\lambda, 1^d, n, t)$: Output a public key pk and secret shares $sk_{i=1}^n$, where d denotes the supported multiplicative depth.
- 2) $c \leftarrow \text{Enc}(m, pk)$: Given a plaintext $m \in \mathbb{Z}_q$ and a public key pk , the algorithm outputs the ciphertext c .
- 3) $\hat{m}_b \leftarrow \text{PDec}(c, sk_b)$: Given a ciphertext c and a partial secret key sk_b , where $b \in [n]$, it outputs the partial decryption $\hat{m}_b$.
- 4) $m \leftarrow \text{TDec}(\hat{m}_1, \hat{m}_2, \dots, \hat{m}_k)$: Given k partial decryption shares $\hat{m}_1, \hat{m}_2, \dots, \hat{m}_k$, where $k \geq t$, the algorithm outputs the decrypted plaintext m .
- 5) $c' \leftarrow \text{Csum}(c_1, c_2)$: Given two ciphertexts c_1 and c_2 , the algorithm outputs a ciphertext encrypting $m_1 + m_2$.
- 6) $c' \leftarrow \text{Cmul}(c_1, c_2)$: Given two ciphertexts c_1 and c_2 , it outputs a ciphertext encrypting $m_1 \times m_2$. (For simplicity, operations such as modulus switching and relinearization are omitted.)
- 7) $\mathcal{R} \leftarrow \text{RotGen}(\{sk_i\}_{i=1}^n)$: Given the set of secret key shares $\{sk_i\}_{i=1}^n$, the algorithm generates rotation keys $\mathcal{R}$ enabling homomorphic slot rotations.
- 8) $c' \leftarrow \text{Crot}(c, k, \mathcal{R})$: Given a ciphertext c , a rotation offset k , and a rotation key set $\mathcal{R}$, the algorithm outputs a ciphertext encrypting the plaintext vector rotated by k positions.

In our work, we pack multiple **IBF** bits into a single ciphertext. During homomorphic computation, we apply rotation operations to align the relevant slots, enabling slot-wise homomorphic additions and multiplications. This packing strategy significantly reduces the overall size of the **EIBF**.

III. SYSTEM MODEL AND THREAT MODEL

This section provides a formal definition of our PMPSI protocol along with the associated threat model.

A. System Model

We now introduce the notion of predicates within the MPSI framework, allowing the intersection operation to be executed only among users who satisfy specific predicate conditions. This

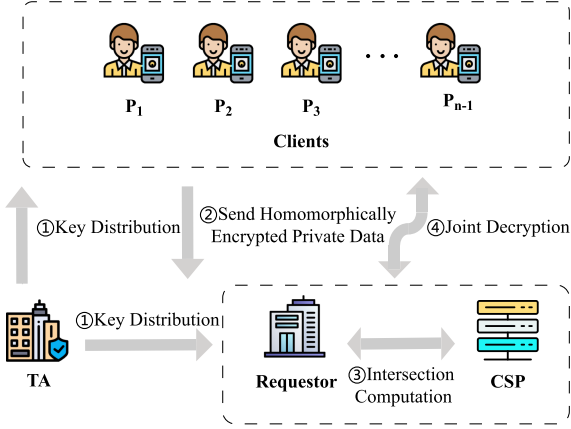


Fig. 2. Construction of PMPSI.

PMPSI Functionality
Parameters: The data set S_i and the attributes $Attr_i = \vec{x}_i$ possessed by each client $P_i, i \in [1, n-1]$, the data set S_n possessed by requestor P_n and the predicate $f = \vec{v}$ defined by requestor P_n. Behavior: Requestor P_n performs the intersection operation with CSP and all clients, then outputs the intersection set $S = \{a_j \mid a_j \in S_{i_1} \cap S_{i_2} \cap \dots \cap S_{i_m}, \text{ s.t. } \forall u \in [1, m], f(Attr_{i_u}) = 1 (\vec{v} \cdot \vec{x}_{i_u} = 0)\}$

Fig. 3. PMPSI functionality.

extended model is referred to as the PMPSI protocol. As shown in Fig. 2, there are four types of entities in our PMPSI protocol:

- **TA:** The TA is a trusted authority that participates only during the setup phase. It is responsible for generating and distributing cryptographic keys, as well as assigning hash functions.
- **Requestor:** We use P_n to denote the requestor in the protocol. The requestor P_n possesses a dataset S_n and defines a predicate $f = \vec{v} = \{v_1, v_2, \dots, v_t\}$.
- **Clients:** We use P_i , where $i \in [1, n-1]$, to denote the clients. Each client P_i holds a set of attributes $Attr_i = \{x_1^i, x_2^i, \dots, x_t^i\}$ and a dataset $S_i = \{a_1^i, a_2^i, \dots, a_m^i\}$.
- **CSP:** The CSP is an auxiliary server that assists the requestor and clients in executing the protocol.

In the PMPSI protocol, the goal of the requestor P_n is to compute the intersection of elements from all clients who satisfy its predicate, without considering whether clients who do not satisfy the predicate also possess those elements. The detailed functionality is illustrated in Fig. 3.

The use of a trusted authority (TA) in our system model is not a fundamental requirement of the proposed protocol. In principle, the TA's functionality can be decentralized using standard distributed key generation techniques under appropriate trust assumptions. In this work, we retain the TA primarily for clarity of presentation and ease of implementation. Notably, the TA only participates in a one-time setup phase and is not involved

in any online computation, and therefore does not impact the online efficiency or robustness of the protocol.

Privacy preservation is a critical aspect of the PMPSI protocol, specifically reflected in the following four dimensions:

- **Element Hiding:** As in standard MPSI protocols, information about elements outside the intersection remains confidential. No party can learn which elements are not part of the intersection.
- **Attribute Hiding:** This property protects client privacy. Neither the requestor nor the CSP can learn any information about the clients' attributes.
- **Predicate Hiding:** This property protects the requestor. Neither the clients nor the CSP can learn any information about the predicate defined by the requestor.
- **Match Hiding:** This property protects the identities of the clients that satisfy the predicate, as well as the exact number of such clients. However, the existence of at least one qualifying client may be inherently revealed by the successful computation of a non-empty intersection. This leakage is an inevitable consequence of the output semantics and does not exceed what is logically implied by the intended functionality.

It is important to emphasize the auxiliary nature of the CSP. In the proposed protocol, the CSP only processes randomized intermediate values and cannot derive any meaningful information about the private inputs. To demonstrate the flexibility of our design, we further introduce a decentralized variant of our protocol in Section IV, where the role of the CSP is distributed among all clients, eliminating the need for a dedicated auxiliary server. The current description retains the CSP as a distinct entity solely for presentational clarity.

B. Threat Model

Our PMPSI protocol operates under the honest-but-curious model, in which all entities (except for the fully trusted TA) follow the protocol honestly but may attempt to infer additional information from the messages they receive. Specifically, the roles are defined as follows: the TA is a fully trusted entity, while the requestor, CSP, and clients are all considered honest-but-curious. Furthermore, we allow collusion among clients, but do not permit any collusion between the requestor or CSP and the clients.

To formally demonstrate the security of the proposed protocols, we define simulators that capture the views of all participating parties in the ideal world. For the DMOC protocol, which involves a requestor, multiple clients, and a CSP, we define three simulators, denoted as $S_{requestor}$, $S_{clients}$, and S_{CSP} , corresponding to the views of the requestor, clients, and CSP, respectively. For the CSP-free variant bDMOC-DC, since no CSP is involved, we define two simulators, $S_{requestor}$ and $S_{clients}$, which simulate the views of the requestor and all participating clients in the decentralized execution. In both settings, we consider an honest-but-curious PPT adversary $\mathcal{A}$. The advantage of $\mathcal{A}$ in distinguishing between the ideal and real executions is defined as

$$Adv_{\Pi}(\mathcal{A}) = |\Pr[\text{View}_{\mathcal{A}}^{\text{Ideal}}(\lambda)] - \Pr[\text{View}_{\mathcal{A}}^{\text{Real}}(\lambda)]|,$$

Where $\Pi \in \{\text{DMOC}, \text{bDMOC-DC}\}$ denotes the protocol under consideration. We say that a protocol Π is secure if $\text{Adv}_{\Pi}(\mathcal{A}) = \text{negl}(\lambda)$, where $\text{negl}(\cdot)$ denotes a negligible function.

In practice, adverse conditions such as network delays, message loss, or participant dropout may lead to premature termination. Such events may affect completeness but do not compromise correctness for completed executions. Importantly, aborts do not introduce additional information leakage, as all intermediate values remain protected by homomorphic encryption and partial transcripts reveal no private inputs. Practical robustness can be addressed orthogonally via standard system-level mechanisms such as retransmissions and restarts, and thus does not affect the security guarantees of PMPSI.

C. Expressiveness of Predicate Representation

We adopt the inner-product predicate formulation [33]. Based on this, our PMPSI model adopts the formulation where a requestor defines a vector $\vec{v}$ and a client with attribute vector $\vec{x}_i$ satisfies the predicate iff $\vec{v} \cdot \vec{x}_i = 0$. This linear design balances expressiveness and cryptographic efficiency by enabling evaluation via additive homomorphic operations. It supports standard encodings such as equality, conjunction, and set membership (via dimension expansion).

We note that this formulation does not directly support arbitrary nonlinear constraints or general Boolean circuits. However, the target applications mainly require equality, membership, and conjunction predicates, which are efficiently captured in our framework. Supporting richer predicates would require feature lifting or more general secure computation (e.g., FHE or MPC), incurring substantially higher overhead. Our design therefore reflects a deliberate trade-off between expressiveness and efficiency.

IV. OUR PMPSI PROTOCOL

In this section, we first introduce the overall design concept of the PMPSI protocol. Next, we present the Dual-Mode Oblivious Comparison (DMOC) protocol, including both the CSP-based construction and its CSP-free variant for different deployment settings. Finally, we provide a detailed description of the complete protocol.

A. Design Rationale and Core Idea

1) *Baseline MPSI With EIBF*: Our PMPSI protocol adopts the design philosophy of Bay's MPSI protocol [9] and utilizes the **Encrypted Inverse Bloom Filter (EIBF)** as its fundamental building block, while further enhancing efficiency by leveraging homomorphic rotation techniques to pack multiple Bloom filter bits into different slots of a single ciphertext, thereby significantly reducing communication overhead. Without incorporating predicates and attributes, the standard MPSI protocol operates in a relatively straightforward manner. One party, denoted as P_n , acts as the server responsible for performing the intersection computation. Each party P_i locally encodes its dataset into an inverse Bloom filter, where a Bloom filter bit corresponding to an inserted element is set to 0, while all other positions

are set to 1. To protect data confidentiality, the inverse Bloom filter is then encrypted using the TFHE scheme, producing an **EIBF**. To improve efficiency, multiple **IBF** bits are packed into different slots of a single ciphertext, allowing each party to transmit a compact EIBF representation to P_n . Upon receiving the EIBFs from all parties, P_n applies each of its elements to the k hash functions (where k denotes the number of hash functions used in the Bloom filter) to compute the corresponding k index positions. For each party P_i , the ciphertext slots corresponding to these k index positions are first homomorphically rotated and aligned to a common slot, after which they are aggregated via homomorphic addition. If an element is also present in party P_i 's dataset, then all k positions in **EIBF** _{i} must encrypt the value 0. Therefore, the ciphertext homomorphic addition of these k positions should also result in a ciphertext of 0, i.e., $C_i = C_{i,1} +_c C_{i,2} +_c \dots +_c C_{i,k} = \text{Enc}_{\text{PHE}}(0)$. Furthermore, if the element exists in the intersection of all parties, then the sum of all C_i values from all parties should still be a ciphertext of 0, i.e., $\text{Value} = C_1 +_c C_2 +_c \dots +_c C_{n-1} = \text{Enc}_{\text{PHE}}(0)$.

2) *Predicate-Supporting Extension*: In this work, we extend the model by introducing predicates, aiming to compute the intersection between P_n and only those parties that satisfy the predicate. In other words, we seek to "filter" out parties that do not meet the predicate requirement. Crucially, this filtering operation must be performed without revealing either the predicate defined by P_n or the attributes of the clients. That is, the filtering process must be conducted entirely in a confidential manner.

Through TFHE scheme, P_n can filter out parties that do not satisfy the predicate, even without knowing which specific parties are excluded. Concretely, we transform each C_i corresponding to a party P_i that does not satisfy the predicate into $\text{Enc}(0)$, regardless of its original value. This is achieved by homomorphically multiplying C_i by a ciphertext of 0. More formally, we generate an auxiliary predicate indicator $F\text{value}_i$ to act as a filter. If P_i satisfies the predicate, then $F\text{value}_i$ is a ciphertext of a non-zero value; otherwise, it is a ciphertext of 0. Ultimately, we compute $\text{Value} = \sum_{i=1}^n (F\text{value}_i \times_c C_i)$, effectively excluding all C_i for parties that do not meet the predicate.

3) *Dual-Mode Transformation*: However, computing each $F\text{value}_i$ confidentially poses a challenge. Each party may send its encrypted attributes to P_n , who can then homomorphically evaluate the predicate function f on the encrypted inputs. According to the definition of inner product-based predicates, $f(\text{Attr}) = 1$ if and only if $\vec{v} \cdot \vec{x} = 0$. Therefore, the output of the predicate function for a qualifying party is a ciphertext of 0. However, for our filtering mechanism, this output must be transformed, i.e., ciphertexts of 0 should become non-zero, and vice versa. Thus, we require a transformation mechanism that allows P_n to invert the predicate result and obtain the desired $F\text{value}$, as illustrated in Fig. 4.

Moreover, during predicate evaluation, only the attributes are encrypted; the predicate f is defined by P_n . As a result, the computation involves only ciphertext addition and ciphertext scalar multiplication, without requiring ciphertext-ciphertext multiplication. Therefore, this operation can be implemented

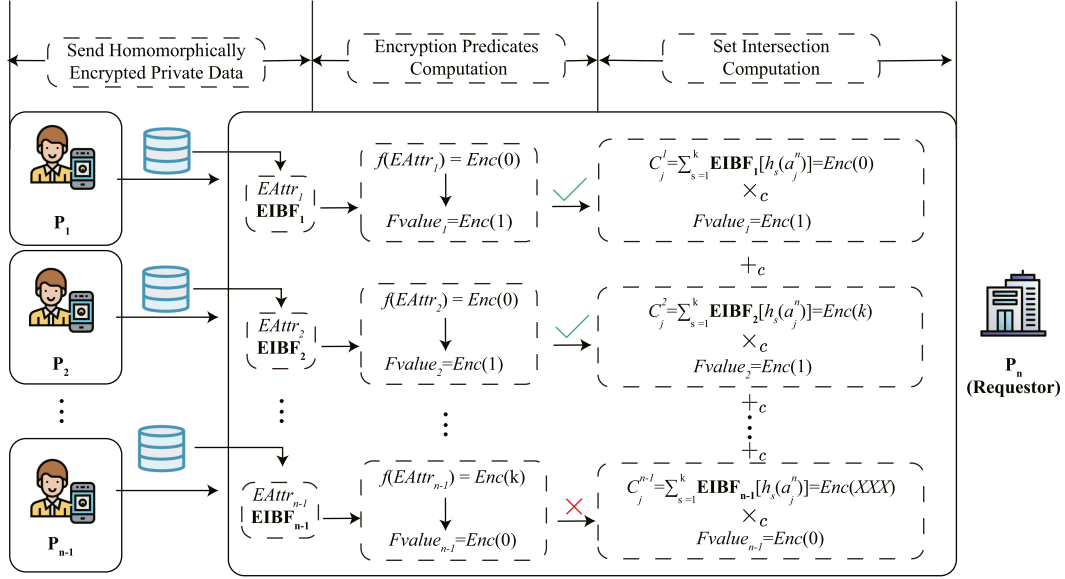


Fig. 4. PMPSI construction overview.

using PHE instead of FHE. Considering the high overhead and key distribution complexity associated with FHE, we adopt PHE for predicate evaluation. Consequently, the aforementioned transformation mechanism must also support conversion from TPHE ciphertexts to TFHE ciphertexts.

To address both transformation requirements, we design the Dual-Mode Oblivious Comparison (DMOC) protocol, which is detailed in Subsection B.

Additionally, we account for the edge case in which no client satisfies the predicate defined by P_n , which could otherwise lead the protocol to yield incorrect results. In such cases, each $Fvalue_i$ for $i \in [1, n-1]$ would be a ciphertext of 0, effectively filtering out all C_i and resulting in a final value of $Enc(0)$. This is clearly incorrect. To handle this situation, we introduce a correction mechanism, the details of which are presented in Subsection C.

B. Dual-Mode Oblivious Comparison Protocol

To address predicate ciphertext transformation, we propose the Dual-Mode Oblivious Comparison (DMOC) protocol, a modification of Zheng's OLTE and OGTE protocols [34]. DMOC converts a non-zero TPHE ciphertext into a TFHE encryption of zero, and vice versa, resolving the semantic mismatch between predicate evaluation and filtering: TPHE yields $m = 0$ for satisfied predicates and $m \neq 0$ otherwise, while filtering requires a zero indicator for non-satisfying clients. The protocol maps the predicate result to a TFHE ciphertext encoding the complementary Boolean condition, enabling correct filtering, and outputs a corresponding TPHE ciphertext as correction information. Executed jointly by the user and CSP under the honest-but-curious model, DMOC relies on random bits B and B' to ensure security: by randomizing the sign of linear combinations, the CSP observes only masked values independent of m , while the user can recover the correct result using B and B' . The full specification is given in Algorithm 1.

As illustrated in Table III, when $m \leq 0$, the value of b_{FHE} is $Enc_{FHE}(1)$; when $m \geq 0$, the value of b'_{FHE} is $Enc_{FHE}(1)$. Therefore, the homomorphic multiplication of b_{FHE} and b'_{FHE} yields $Enc_{FHE}(1)$ if and only if $m = 0$. In all other cases (i.e., when $m \neq 0$), the result of their homomorphic multiplication is $Enc_{FHE}(0)$. The variable cor represents the partially homomorphic ciphertext sum of b_{PHE} and b'_{PHE} , serving as a correction bit, as shown in Table IV. The specific use of this correction information will be further elaborated in Subsections C and D.

In addition to security, the correctness of the comparison relies on appropriate parameter settings. The random masking induced by B and B' must preserve the sign of m , requiring that randomization and noise growth remain within bounds. Under standard TPHE and TFHE parameters, this holds as long as $|m|$ lies within the valid message space and noise stays below the decryption threshold. Thus, comparisons ($m \leq 0$, $m \geq 0$) are correct with overwhelming probability, while $m = 0$ is handled by the dual comparison structure. Improper parameter choices (e.g., excessive noise) may break correctness but are excluded by construction.

In real-world scenarios, clients may not fully trust the requestor, potentially suspecting that the requestor could choose a CSP that favors its own interests. To further strengthen privacy guarantees and accommodate decentralized deployment scenarios, we introduce a CSP-free scheme bDMOC-DC (Batch-enabled Decentralized DMOC), in which all clients collaboratively execute the computational tasks. As presented in Algorithm 2, all operations are performed on data encrypted under the TFHE scheme. Clients proceed sequentially according to their assigned indices, with each participant independently generating a permutation π_i to enhance privacy by breaking positional linkage between ciphertexts across protocol rounds. During the protocol, every client successively performs partial decryption, permutation, and re-randomization. These operations ensure that intermediate ciphertexts cannot be linked across

Algorithm 1: DMOC Protocol.

- 1: **Input:** User inputs the PHE ciphertext $Enc_{PHE}(m)$ and holds $TPHE.sk_1$ (for partial decryption), CSP holds $TPHE.sk_2$.
- 2: **Output:** The transformed FHE ciphertext $Fvalue$ and the correction information cor .
- 3: *User:*
- 4: selects $r_1, r_2, r'_1, r'_2 \leftarrow \mathbb{Z}_N$ randomly s.t. $r_1 > r_2 > 0, r'_1 > r'_2 > 0$
- 5: selects two random bits $B, B' \in \{-1, 1\}$. Prevent the CSP from learning any information about m through the decryption process
- 6: $c = B \times_c (r_1 \times_c Enc_{PHE}(m) -_c Enc_{PHE}(r_2))$
- 7: $c' = B' \times_c (r'_1 \times_c Enc_{PHE}(m) +_c Enc_{PHE}(r'_2))$
- 8: partially decrypts c, c' , obtaining pd_c, pd_c'
- 9: sends $(c, pd_c), (c', pd_c')$ to CSP
- 10: *CSP:*
- 11: fully decrypts $(c, pd_c), (c', pd_c')$, obtaining $\bar{m}, \bar{m}'$.
- 12: $\bar{b}_{PHE}, \bar{b}_{FHE} = \begin{cases} (Enc_{PHE}(1), Enc_{FHE}(1)) & \text{if } \bar{m} \leq 0 \\ (Enc_{PHE}(0), Enc_{FHE}(0)) & \text{otherwise} \end{cases}$
- 13: $\bar{b}'_{PHE}, \bar{b}'_{FHE} = \begin{cases} (Enc_{PHE}(1), Enc_{FHE}(1)) & \text{if } \bar{m}' \geq 0 \\ (Enc_{PHE}(0), Enc_{FHE}(0)) & \text{otherwise} \end{cases}$
- 14: sends $(\bar{b}_{PHE}, \bar{b}_{FHE}, \bar{b}'_{PHE}, \bar{b}'_{FHE})$ to User.
- 15: *User:*
- 16: **if** $B == 1$: $b_{PHE}, b_{FHE} = \bar{b}_{PHE}, \bar{b}_{FHE}$
 $b_{PHE} = Enc_{PHE}(1) +_c (-1 \times_c \bar{b}_{PHE})$
- 17: **else:** $b_{FHE} = Enc_{FHE}(1) +_c (-1 \times_c \bar{b}_{FHE})$
- 18: **if** $B' == 1$: $b'_{PHE}, b'_{FHE} = \bar{b}'_{PHE}, \bar{b}'_{FHE}$
 $b'_{PHE} = Enc_{PHE}(1) +_c (-1 \times_c \bar{b}'_{PHE})$
- 19: **else:** $b'_{FHE} = Enc_{FHE}(1) +_c (-1 \times_c \bar{b}'_{FHE})$
- 20: $Fvalue = b_{FHE} \times_c b'_{FHE}$ $cor = b'_{PHE} +_c b_{PHE}$

TABLE III
Fvalue VALUE BASED ON DIFFERENT SCENARIOS

scenario	$m = 0$	$m > 0$	$m < 0$
b_{FHE}	$Enc_{FHE}(1)$	$Enc_{FHE}(0)$	$Enc_{FHE}(1)$
b'_{FHE}	$Enc_{FHE}(1)$	$Enc_{FHE}(1)$	$Enc_{FHE}(0)$
$Fvalue$	$Enc_{FHE}(1)$	$Enc_{FHE}(0)$	$Enc_{FHE}(0)$

rounds, preventing inference based on ordering or structural consistency. After the sequence completes, client P_{n-1} carries out the full decryption and executes the operations originally assigned to the CSP in Algorithm 2. The resulting ciphertext is then propagated in reverse order through the client chain, thereby restoring the original data sequence.

The security of bDMOC-DC no longer relies on the “non-collusion” assumption between the requestor and a CSP. Instead, privacy is guaranteed by a distributed permutation mechanism. Specifically, each client’s private data is iteratively

TABLE IV
cor VALUE BASED ON DIFFERENT SCENARIOS

scenario	$m = 0$	$m > 0$	$m < 0$
b_{PHE}	$Enc_{PHE}(1)$	$Enc_{PHE}(0)$	$Enc_{PHE}(1)$
b'_{PHE}	$Enc_{PHE}(1)$	$Enc_{PHE}(1)$	$Enc_{PHE}(0)$
cor	$Enc_{PHE}(2)$	$Enc_{PHE}(1)$	$Enc_{PHE}(1)$

Algorithm 2: bDMOC-DC Protocol.

- 1: **Input:** User inputs the FHE ciphertext $\mathbf{X} = (Enc_{FHE}(m_1), \dots, Enc_{FHE}(m_{n-1}))$ and the secret key $TFHE.sk_1$; Each client $Client_i$ holds a secret key $TFHE.sk_i$ and a permutation π_i .
- 2: **Output:** The transformed FHE ciphertext $(Fvalue_1, \dots, Fvalue_{n-1})$ and the correction information $(cor_1, \dots, cor_{n-1})$.
- 3: *User:*
- 4: computes $\mathbf{X}_0 = \{(c_t, pd_{c_t}), (c'_t, pd_{c'_t}) \mid t = 1, \dots, n-1\}$ using the TFHE-based procedure specified in Algorithm 1
- 5: sends $(c_t, pd_{c_t}), (c'_t, pd_{c'_t})$ to Clients
- 6: *Clients:*
- 7: **for** $i = 1$ to $n-1$ **do**
- 8: Send $\mathbf{X}_{i-1}$ to P_i
- 9: $\mathbf{X}_i \leftarrow \text{Shuffle}(\mathbf{X}_{i-1}, \pi_i)$. Randomly permute the ciphertext blocks
- 10: $\mathbf{X}_i \leftarrow \text{Re-Random}(\mathbf{X}_i)$.
- 11: **end for**
- 12: $\text{JointDecrypt}(\{P_1, \dots, P_{n-1}\}, \mathbf{X}_{n-1}, \{TFHE.sk_i\}_{i=1}^{n-1})$
- 13: computes $\mathbf{B}_n = \{\bar{b}_{FHE,t}, \bar{b}'_{PHE,t} \mid t = 1, \dots, n-1\}$ using the procedure specified in Algorithm 1
- 14: **for** $i = n-1$ to 1 **do**
- 15: Send $\mathbf{B}_{i+1}$ to P_i
- 16: $\mathbf{B}_i \leftarrow \text{Re-Shuffle}(\mathbf{B}_{i+1}, \pi_i)$. Shuffle Permutation Restoration.
- 17: **end for**
- 18: *User:*
- 19: gets $\mathbf{B}_1$ from P_1 , and computes $Fvalue$ and cor using the TFHE-based procedure specified in Algorithm 1.

obscured by all preceding permutations π_i , mapping it to an indistinguishable ciphertext location and preventing adversaries from tracking element positions across protocol steps. Even though the last client P_{n-1} performs the computation equivalent to the CSP, it cannot associate any ciphertext with a particular client’s private input without knowledge of every π_i . Consequently, a single client cannot infer other participants’ privacy through collusion with any party (including the requestor); moreover, joining a collusion could risk exposing its own permuted data. Without these permutation and re-randomization steps, ciphertext ordering would remain stable across rounds, enabling linkage attacks that allow adversaries to correlate inputs with outputs or trace data across protocol steps. This

design effectively prevents such linkage and collusion-based inference attacks while preserving the feasibility of distributed computation.

The security of the DMOC protocol will be analyzed in Section V, together with the security analysis of the PMPSI protocol.

C. Concrete Construction

The details of our PMPSI protocol are as follows:

Input: Each client $P_i, i \in [1, n-1]$, possesses a set of attributes $Attr_i = \{x_1^i, x_2^i, \dots, x_t^i\}$ and a dataset $S_i = \{a_1^i, a_2^i, \dots, a_m^i\}$. The requestor P_n defines a predicate $f = \bar{v} = \{v_1, v_2, \dots, v_t\}$ and holds its own dataset $S_n = \{a_1^n, a_2^n, \dots, a_m^n\}$. Additionally, P_n initializes an empty intersection set S , and let s be the number of IBF bits to be packed into a single plaintext vector.

Setup: A trusted authority (TA) generates and distributes the cryptographic keys and system parameters as follows:

- 1) The TA then invokes TFHE.KeyGen to generate a public key TFHE.pk and n private keys: $\text{TFHE.sk}_1, \text{TFHE.sk}_2, \dots, \text{TFHE.sk}_n$. It broadcasts TFHE.pk to all parties and sends TFHE.sk_i to the corresponding participant P_i , for all $i \in [1, n]$.
- 2) In the DMOC protocol, the TA invokes TPHE.KeyGen to generate a public key TPHE.pk and two private keys TPHE.sk_1 and TPHE.sk_2 . The TA broadcasts TPHE.pk to all entities in the protocol, sends TPHE.sk_1 to the requestor P_n (which will be used for partial decryption in the DMOC protocol), and sends TPHE.sk_2 to the CSP. In the bDMOC-DC protocol, the TA computes a set of homomorphic rotation keys $\mathcal{R}$ using the function TFHE.RotGen . It then securely transmits the rotation key set $\mathcal{R}$ to the requestor P_n , enabling P_n to perform homomorphic slot rotations during encrypted vector processing.
- 3) The TA also specifies k hash functions $\{h_1, h_2, \dots, h_k\}$ and distributes them to all clients and the requestor.

Encryption of Attributes and EIBF Generation: Each client $P_i, i \in [1, n-1]$, performs the following operations:

- 1) P_i encrypts each component of the attribute vector using the public key TPHE.pk from the TPHE scheme, obtaining a set of ciphertexts for the attribute vector, denoted as $EAttr_i = \{\text{Enc}(x_1^i), \text{Enc}(x_2^i), \dots, \text{Enc}(x_t^i)\}$.¹
- 2) P_i generates its Bloom Filter $\mathbf{BF}_i$ using the hash functions $\{h_1, h_2, \dots, h_k\}$ and its dataset S_i . Then, P_i transforms $\mathbf{BF}_i$ into $\mathbf{IBF}_i$ and employs a packing strategy to group multiple bits of $\mathbf{IBF}_i$ into a single plaintext vector. Each packed vector is then encrypted under the public key TFHE.pk , resulting in the encrypted packed Bloom Filter:

$$\mathbf{EIBF}_i = \left\{ c_i^{(1)}, c_i^{(2)}, \dots, c_i^{(\lceil |\mathbf{IBF}_i|/s \rceil)} \right\}.$$

Within each ciphertext $c_i^{(j)}$, the s bits occupy individual slots of the plaintext vector.

- 3) P_i sends $EAttr_i$ and $\mathbf{EIBF}_i$ to P_n .

Encrypted Predicate Computation: For each $i \in [1, n-1]$, the requestor P_n performs the following operations in parallel with the CSP:

- 1) P_n computes the “predicate value” in the form of ciphertexts as follows:

$$\begin{aligned} fvalue_i &= f(EAttr_i) \\ &= \sum_{j=1}^t (v_j \times_c \text{Enc}(x_j^i))^1 \end{aligned}$$

- 2) P_n acts as the user and runs the DMOC protocol with the CSP, taking $fvalue_i$ as input, and obtains the transformed ciphertext output $Fvalue_i$ along with the correction information cor_i .

Then P_n computes the aggregated correction value $fcor$ as follows:

$$fcor = \left(\sum_{i=1}^{n-1} cor_i \right) -_c \text{Enc}(n-1)^1$$

Using $fcor$, P_n then performs an additional oblivious comparison with the CSP to obtain $Fcor$ (where $Fcor = b_{FHE}$).

Set Intersection Computation: For each element $a_j^n, j \in [1, m]$, of the requestor P_n , the following operations are performed:

- 1) P_n uses the set of hash functions $\{h_1, h_2, \dots, h_k\}$ to compute a sequence of index values $\{h_1(a_j^n), h_2(a_j^n), \dots, h_k(a_j^n)\}$.
- 2) For each $\mathbf{EIBF}_i, i \in [1, n-1]$, P_n retrieves the ciphertexts at the corresponding index positions, resulting in $\{\mathbf{EIBF}_i[h_1(a_j^n)], \mathbf{EIBF}_i[h_2(a_j^n)], \dots, \mathbf{EIBF}_i[h_k(a_j^n)]\}$, and computes C_j^i as follows:

$$C_j^i = \sum_{s=1}^k \mathbf{EIBF}_i[h_s(a_j^n)]$$

- 3) For each $\mathbf{EIBF}_i, i \in [1, n-1]$, and for each hash position $h_t(a_j^n)$, P_n first identifies the ciphertext containing the slot corresponding to $h_t(a_j^n)$. Then, it applies the homomorphic rotation function $\mathbf{Crot}$ to move the target slot within the ciphertext to the first position. The aggregated ciphertext C_j^i is computed as

$$\begin{aligned} C_j^i &= \sum_{t=1}^k \mathbf{Crot}(\mathbf{EIBF}_i[\lceil h_t(a_j^n)/s \rceil], \\ &\quad h_t(a_j^n) \bmod s, \mathcal{R}), \end{aligned}$$

where $\mathcal{R}$ is the set of rotation keys.

- 4) P_n computes $Value_j$ as:

$$Value_j = \sum_{i=1}^{n-1} (Fvalue_i \times_c C_j^i) +_c Fcor$$

The ciphertext $Value_j$ represents the aggregated indicator determining whether the element a_j^n belongs to the intersection. Subsequently, a homomorphic perturbation

¹In the DMOC protocol, Enc refers to the encryption function under the PHE scheme, while in the bDMOC-DC protocol it refers to the FHE scheme.

TABLE V
EXAMPLE OF THE CORRECTION MECHANISM

	P_1	P_2	P_3	f_{cor}	Result	Scenario
$Fvalue_i$	$Enc(1)$	$Enc(1)$	$Enc(0)$			
C_j^i	$Enc(0)$	$Enc(0)$	$Enc(?)$	$Enc(2)$	$Enc(0)$	In the intersection
$Fcor$	$Enc(2)$	$Enc(2)$	$Enc(1)$			
$Fvalue_i$	$Enc(1)$	$Enc(0)$	$Enc(1)$			
C_j^i	$Enc(0)$	$Enc(?)$	$Enc(k)$	$Enc(2)$	$Enc(k)$	Not in the intersection
$Fcor$	$Enc(2)$	$Enc(1)$	$Enc(2)$			
$Fvalue_i$	$Enc(0)$	$Enc(0)$	$Enc(0)$			
C_j^i	$Enc(?)$	$Enc(?)$	$Enc(?)$	$Enc(0)$	$Enc(1)$	No client satisfies
$Fcor$	$Enc(1)$	$Enc(1)$	$Enc(1)$			

Note. $Enc(k)$ denotes the encryption of a positive integer. $Enc(?)$ denotes the encryption of an irrelevant value that has no impact on the final result.

is applied to $Value_j$ to protect individual privacy before performing threshold decryption.

- 5) To perturb $Value_j$, P_n selects a random non-zero number $r_{refresh} \leftarrow \mathbb{Z}_q$ and computes:

$$Value'_j = Enc_{FHE}(r_{refresh}) \times_c Value_j$$

- 6) P_n randomly selects a client P_m and sends $Value'_j$ to P_m .
- 7) P_m encrypts a random non-zero number r_m and further perturbs $Value'_j$ as follows:

$$PValue_j = Enc_{FHE}(r_m) \times_c Value'_j$$

Then, P_m broadcasts $PValue_j$ to all clients and the requestor $\{P_1, P_2, \dots, P_n\}$.

- 8) All clients use their partial secret keys $TFHE.sk_i$ to perform partial decryption using $TFHE.PDec$ and send the results to P_n .
- 9) Finally, P_n uses the recovery algorithm $TFHE.TDec$ to decrypt the final result. If the result is 0, then the element a_j^n is added to the intersection set S .

Output: The intersection set S

D. Discussion

Based on the concrete construction of the PMPSI protocol presented in Subsection C, it is evident that the encrypted predicate computation is independent of the intersection operations performed on the requestor's dataset. This indicates that predicate computation needs to be performed only once. Thereafter, the results can be stored in a cache and reused whenever the requestor needs to compute the intersection for elements in their dataset.

We now elaborate on the correction mechanism, which is critical for ensuring both correctness and output indistinguishability in extreme cases. Specifically, the correction information is designed to prevent the final computation result from degenerating into a structurally invalid or ambiguous encoding, a ciphertext of 0 in the scenario where no client satisfies the requestor's predicate. The DMOC protocol outputs a correction bit ciphertext cor_i in TPHE ciphertext form, which is used to adjust the result in such edge cases.

As shown in Table V, the protocol has three scenarios: (1) at least one client satisfies the predicate and the element is in the intersection; (2) at least one client satisfies the predicate but

the element is not in the intersection; (3) no client satisfies the predicate. Only the third case requires correction, as it would otherwise yield an all-zero ciphertext vector, leading to invalid decoding. In this case, all cor_i encrypt 1, and their homomorphic sum equals $Enc(n-1)$. By subtracting $Enc_{FHE}(n-1)$ homomorphically, the result is a ciphertext of 0. In all other scenarios, the sum will differ from $n-1$, and the subtraction yields a ciphertext greater than 0. An oblivious comparison then maps $Enc(0) \rightarrow Enc(1)$ and non-zero ciphertexts to $Enc(0)$, producing $Fcor$. Finally, $Fcor$ is added to the intersection result: it has no effect in the first two cases, but in the third case introduces $Enc(1)$, preventing a zero output. However, this mechanism may leak the extreme case when the decrypted result is 1, which is mitigated by a randomly selected client adding perturbation during joint decryption to mask the value and preserve privacy.

We further note that the execution behavior of the protocol is largely insensitive to the proportion of predicate-satisfying clients. The dominant computational and communication costs depend primarily on the total number of participants and the input size, rather than on the number of matches. Although the error-correction and filtering mechanisms may follow slightly different execution paths depending on intermediate results, they are applied uniformly over the full set of clients and candidate elements. As a result, this invariance also provides a side-channel security benefit by preventing an adversary from inferring the satisfaction ratio through timing or execution patterns.

V. CORRECTNESS AND SECURITY ANALYSIS

In this section, we analyze the correctness of our PMPSI protocol and demonstrate its security under the assumed threat model.

A. Correctness

In the PMPSI protocol, the operations performed by the requestor P_n on each element a_j^n , $j \in [1, m]$, in its dataset S_n are identical. Therefore, it suffices to demonstrate the correctness for a single element with respect to its inclusion in the intersection.

In the extreme case where no clients satisfy the predicate defined by P_n , all $Fvalue_i$ values will be $Enc_{FHE}(0)$, and $Fcor$ will be $Enc_{FHE}(1)$. In this case, $Value_j$ is computed as follows:

$$\begin{aligned} Value_j &= \sum_{i=1}^{n-1} (Enc_{FHE}(0) \times_c C_j^i) +_c Enc_{FHE}(1) \\ &= Enc_{FHE}(1) \end{aligned}$$

Therefore, the result, once perturbed by both P_n and P_m , becomes a random non-zero ciphertext.

Next, we consider the general case. Let $\mathcal{R}$ denote the set of clients that satisfy the requestor's predicate, and $\mathcal{U}$ the set of clients that do not. In this scenario, $Fcor$ is equal to $Enc_{FHE}(0)$. Accordingly, $Value_j$ is computed as:

$$Value_j = \sum_{P_i \in \mathcal{R}} (Enc_{FHE}(1) \times_c C_j^i)$$

$$\begin{aligned}
& +_c \sum_{P_i \in \mathcal{U}} (Enc_{FHE}(0) \times_c C_j^i) +_c Enc_{FHE}(0) \\
& = \sum_{P_i \in \mathcal{R}} (Enc_{FHE}(1) \times_c C_j^i)
\end{aligned}$$

If the element a_j^n belongs to the intersection of the clients in $\mathcal{R}$, it must appear in the **EIBF** of each of those clients. As a result, for each $P_i \in \mathcal{R}$, the corresponding C_j^i is $Enc_{FHE}(0)$, and thus:

$$Value_j = \sum_{P_i \in \mathcal{R}} (Enc_{FHE}(1) \times_c Enc_{FHE}(0)) = Enc_{FHE}(0)$$

When $Value_j = Enc_{FHE}(0)$, the final result remains unchanged by the perturbations applied by P_n and the selected client P_m :

$$\begin{aligned}
PValue_j &= Enc_{FHE}(r_{refresh}) \times_c Enc_{FHE}(r_m) \\
&\times_c Enc_{FHE}(0) \\
&= Enc_{FHE}(0)
\end{aligned}$$

B. Security

We now prove that the specific construction presented in Section IV satisfies the formal security definition of the PMPSI protocol as defined in Section III, by establishing the following theorems.

Theorem 1: In the honest-but-curious model, there exists a simulator S_{P_n} that can simulate the ideal view, which is indistinguishable from the real view of P_n to an adversary $\mathcal{A}$, such that:

$$\text{Sim}_{\mathcal{A}}^{S_{P_n}}(\lambda) \equiv \text{REAL}_{\mathcal{A}}^{P_n}(\lambda)$$

provided that the TPHE and TFHE schemes are IND-CPA secure.

Proof: We use the standard hybrid argument to prove Theorem 1. We construct three hybrids that gradually modify the real protocol until it becomes fully simulated.

Hybrid 1: In this hybrid, the simulator S_{P_n} simulates only the DMOC protocol with the CSP. Specifically, it must simulate $(\bar{b}_{PHE}, \bar{b}_{FHE}, \bar{b}'_{PHE}, \bar{b}'_{FHE})$ returned by the CSP. The simulation proceeds as follows:

- 1) S_{P_n} selects four random values $r_1^{sim}, r_2^{sim}, r_1'^{sim}, r_2'^{sim} \leftarrow \mathbb{Z}_N$ such that $r_1^{sim} > r_2^{sim} > 0$ and $r_1'^{sim} > r_2'^{sim} > 0$, and selects two random bits $B^{sim}, B'^{sim} \in \{-1, 1\}$. Then, using $TPHE.pk$, it computes:

$$\begin{aligned}
c^{sim} &= B^{sim} \times_c (r_1^{sim} \times_c fvalue_i -_c Enc_{PHE}(r_2^{sim})) \\
c'^{sim} &= B'^{sim} \times_c (r_1'^{sim} \times_c fvalue_i +_c Enc_{PHE}(r_2'^{sim}))
\end{aligned}$$

Next, S_{P_n} uses $TPHE.sk_1$ to perform partial decryption, obtaining $(c^{sim}, pd_{c^{sim}})$ and $(c'^{sim}, pd_{c'^{sim}})$.

- 2) If $B^{sim} = 1$, then S_{P_n} sets $\bar{b}_{PHE} = \bar{b}'_{PHE} = Enc_{PHE}(0)$; if $B^{sim} = -1$, then both are set to $Enc_{PHE}(1)$. The values $\bar{b}_{FHE}^{sim}$ and $\bar{b}'_{FHE}^{sim}$ are simulated in the same way based on B'^{sim} .

In the DMOC protocol, if $b_{PHE} = b'_{PHE} = Enc_{PHE}(0)$ and $b_{FHE} = b'_{FHE} = Enc_{FHE}(0)$, then the simulation matches exactly. If instead, any of the values as $Enc_{PHE}(1)$ or $Enc_{FHE}(1)$, and an adversary $\mathcal{A}$ can distinguish these from their simulated counterparts, then $\mathcal{A}$ can break the IND-CPA security of TPHE or TFHE.

In the bDMOC-DC protocol, the remaining clients jointly perform the functionality originally handled by the CSP. The simulator S_{P_n} receives ciphertexts that are transformed from TPHE encryptions into TFHE encryptions, while the original TFHE ciphertexts remain unchanged. Since this transformation only re-encrypts data under an IND-CPA secure scheme and preserves the cryptographic freshness of the ciphertexts, Hybrid 1 is computationally indistinguishable from the real protocol execution.

Hybrid 2: In this hybrid, S_{P_n} also simulates the predicate value $fvalue_i$ using a random number:

- 1) It selects $r_1^{sim}, r_2^{sim}, r_1'^{sim}, r_2'^{sim} \leftarrow \mathbb{Z}_N$ such that $r_1^{sim} > r_2^{sim} > 0$ and $r_1'^{sim} > r_2'^{sim} > 0$, and chooses $B^{sim}, B'^{sim} \in \{-1, 1\}$.
- 2) It samples $fvalue_i^{sim} \leftarrow \mathbb{Z}_N$ and computes:

$$c^{sim} = B^{sim} \times_c (r_1^{sim} \times_c fvalue_i^{sim} -_c Enc_{PHE}(r_2^{sim}))$$

$$c'^{sim} = B'^{sim} \times_c (r_1'^{sim} \times_c fvalue_i^{sim} +_c Enc_{PHE}(r_2'^{sim}))$$

Since $fvalue_i^{sim}$ is a random ciphertext, if $\mathcal{A}$ can distinguish Hybrid 2 from Hybrid 1, then it breaks IND-CPA security of TPHE. Thus, Hybrid 1 and Hybrid 2 are indistinguishable.

Hybrid 3: In this final hybrid, S_{P_n} also simulates **IBF**_{*i*} as a random bit string, encrypts it using $TFHE.Enc$, and obtains **EIBF**_{*i*}^{sim}. All other operations follow Hybrid 2.

If an adversary $\mathcal{A}$ can distinguish Hybrid 3 from Hybrid 2, then they break the IND-CPA security of TFHE. Hence, Hybrid 2 and Hybrid 3 are also indistinguishable.

Since Hybrid 3 fully simulates P_n 's view.

Theorem 2: In the honest-but-curious model, there exists a simulator S_{CSP} that can simulate the ideal view, indistinguishable from CSP's real view to an adversary $\mathcal{A}$, such that:

$$\text{Sim}_{\mathcal{A}}^{S_{CSP}}(\lambda) \equiv \text{REAL}_{\mathcal{A}}^{CSP}(\lambda)$$

assuming reliable random number selection.

Proof: Since the CSP only participates in the oblivious comparison protocol, S_{CSP} only needs to simulate $(c, pd_{c'})$ and $(c', pd_{c'})$ as sent by P_n in the real execution. To do this, S_{CSP} selects two random values $value_1^{sim}$ and $value_2^{sim}$, computes $c^{sim} = Enc_{PHE}(value_1^{sim})$ and $c'^{sim} = Enc_{PHE}(value_2^{sim})$, and then performs partial decryption to obtain $pd_{c^{sim}}$ and $pd_{c'^{sim}}$.

Due to the randomness in real ciphertexts (e.g., randomness r_1, r_2, B), $\mathcal{A}$ cannot distinguish the simulation from the real view.

Theorem 3: In the honest-but-curious model, there exists a simulator S_{P_i} that can simulate the ideal view, indistinguishable from P_i 's real view to an adversary $\mathcal{A}$, such that:

$$\text{Sim}_{\mathcal{A}}^{S_{P_i}}(\lambda) \equiv \text{REAL}_{\mathcal{A}}^{P_i}(\lambda)$$

provided that the TFHE scheme is IND-CPA secure.

Proof: In the DMOC protocol, each client $P_i, i \in [1, n - 1], i \neq m$ only receives the perturbed ciphertext $PValue_j$ from a random client P_m at the end of the protocol. P_m receives $Value'_j$ directly from P_n . The simulator S_{P_i} simply simulates both of these by selecting a random value r_i and computing $PValue_j^{sim} = Enc_{FHE}(r_i)$. Since these ciphertexts are fresh and IND-CPA secure, $\mathcal{A}$ cannot distinguish them from the real ones.

The bDMOC-DC protocol follows the same structure with additional overhead: the last client executes DMOC-equivalent operations, while others perform only shuffling and re-randomization. As re-randomization preserves ciphertext freshness and shuffling only permutes order, no additional information is leaked.

Conclusion: We have constructed simulators for all three semi-honest parties— P_n , CSP, and P_i —such that their ideal and real views are indistinguishable. Therefore, we conclude that:

$$|\Pr(\text{View}_{\mathcal{A}}^{Ideal}(\lambda)) - \Pr(\text{View}_{\mathcal{A}}^{Real}(\lambda))| = \text{negl}(\lambda)$$

proving that the PMPSI protocol meets the security definition given in Section III.

VI. EXPERIMENTS

In this section, we present the implementation and evaluation of our protocol using C++ to demonstrate its performance.

A. Experiment Setting

Our implementation uses the OpenFHE, GMP, and Bloom Filter libraries. OpenFHE enables TFHE construction based on the BGV framework [35], while GMP supports TPHE implementation; the Bloom Filter library is used to generate client filters. For the RLWE-based TFHE scheme, we set the ring dimension to 8,192 and enable multi-party mode with noise flooding to enhance privacy. For TPHE, the master key length is 512 bits.

To ensure that the false positive rate of the Bloom filter (p) remains below 1%, we compute the parameters based on the *total number of elements across all clients and the requestor*, denoted by n . The size of the bit array m is calculated as:

$$m = -\frac{n \cdot \ln(p)}{(\ln(2))^2}$$

The optimal number of hash functions k can then be determined by:

$$k = \frac{m}{n} \ln 2$$

In our experiments, n is dynamically set per configuration, and the Bloom filter parameters m and k are recomputed to maintain a false positive rate below 1%. As these depend only on the aggregate input size, filtering performance remains unaffected by data distribution across clients, naturally supporting highly imbalanced datasets.

The protocol is implemented using standard cryptographic libraries without specialized hardware or low-level optimizations; parallelization is applied only in the set intersection step, where

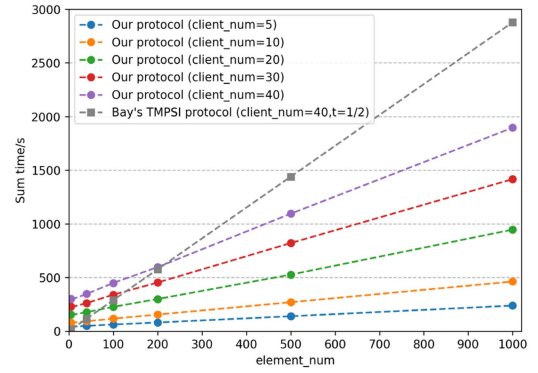


Fig. 5. Total runtime at *predicate_size* = 16.

independent element-wise operations are executed concurrently. Other components remain sequential, so the reported performance reflects the intrinsic efficiency of the protocol design.

B. PMPSI Elements Number Testing

Table VI presents the runtime results of our protocol. We use “Setup”, “Step 1”, “Step 2”, and “Step 3” to represent the phases of “Key Generation and Distribution”, “Encryption Attributes and EIBF Generation”, “Encrypted Predicate Computation”, and “Set Intersection Computation” in the PMPSI protocol, respectively. We fix the length of the vector $\vec{v}$ representing the predicate defined by the requestor (*predicate_size*) at 16. The unit of measurement for runtime is seconds. In particular, *element_num* refers to the number of elements owned by every Participant, and *client_num* denotes the number of participating clients.

Due to our use of TFHE, the setup computational cost grows approximately linearly with the number of clients. Compared with the original DMOC protocol, bDMOC-DC removes the CSP’s assistance and consequently shifts additional computation onto the protocol parties (or the coordinator), producing noticeably higher computational overhead; nonetheless, this overhead remains acceptable in our experiments given the security and deployment trade-offs introduced by eliminating the trusted helper. Step 3 constitutes the dominant portion of the computational cost, as it performs per-element, per-participant computations. Consequently, its runtime exhibits a near-linear growth influenced jointly by both the *client_num* and *element_num*.

Our scheme is designed to allocate computational tasks efficiently: the TA manages Setup, and Requestor executes the more demanding Steps 2 and 3. This leaves the client responsible only for the least expensive operations in Step 1. As TA and Pn are typically provisioned with greater computational power in real-world deployments, this architecture results in markedly enhanced practicality and client efficiency.

In addition, we compare our PMPSI protocol with Bay’s threshold multi-party private set intersection (TMPSI) protocol [9], as it is the only known protocol that offers similar functionality. To a certain extent, our protocol can be viewed as a fine-grained extension of Bay’s TMPSI. As shown in Fig. 5, Bay’s protocol has an advantage when the number of elements

TABLE VI
RUNTIME RESULTS (IN SECONDS) FOR OUR SCHEME WHEN THE *predicate_size* IS SET TO 16. *client_num* DENOTES THE NUMBER OF PROTOCOL PARTICIPANTS, AND *element_num* DENOTES THE AVERAGE SET SIZE PER PARTY.

element_num	client_num	DMOC		bDMOC-DC		Step3(s)	element_num	client_num	DMOC		bDMOC-DC		Step3(s)
		Setup(s)	Step1(s)	Step2(s)	Step1(s)				Setup(s)	Step1(s)	Step2(s)	Step1(s)	
5	5	38.94	0.05	1.30	0.49	3.33	100	5	38.76	0.25	1.23	0.68	20.8
5	10	72.24	0.05	2.35	0.48	7.11	100	10	72.63	0.25	2.05	0.68	40.8
5	20	139.4	0.05	4.37	0.48	19.1	100	20	139.9	0.25	4.56	0.68	81.2
5	30	207.1	0.05	6.55	0.48	35.5	100	30	208.3	0.25	6.75	0.68	123
5	40	273.5	0.05	9.04	0.48	56.5	100	40	273.3	0.25	8.88	0.68	165
10	5	38.86	0.05	1.16	0.48	3.16	200	5	38.74	0.45	1.32	0.89	39.5
10	10	72.55	0.05	2.34	0.48	7.30	200	10	72.46	0.45	2.22	0.88	78.7
10	20	139.2	0.05	4.74	0.48	19.4	200	20	139.9	0.45	4.27	0.88	154
10	30	206.9	0.05	6.57	0.48	35.4	200	30	207.7	0.45	7.29	0.88	236
10	40	273.4	0.05	8.50	0.48	56.7	200	40	274.3	0.45	9.03	0.90	312
20	5	38.84	0.07	1.16	0.51	3.19	500	5	38.93	1.10	1.01	1.53	97.2
20	10	72.32	0.07	2.36	0.51	7.29	500	10	72.45	1.10	2.29	1.53	193
20	20	140.0	0.07	4.37	0.51	19.3	500	20	139.3	1.10	4.70	1.54	380
20	30	207.5	0.07	7.07	0.51	35.4	500	30	207.7	1.11	7.16	1.60	604
20	40	273.7	0.07	9.54	0.51	56.8	500	40	274.7	1.13	9.68	1.62	808
40	5	38.82	0.10	1.13	0.54	3.10	1000	5	39.07	2.18	1.13	2.58	194
40	10	72.31	0.10	2.48	0.54	7.57	1000	10	72.40	2.14	2.17	2.58	385
40	20	140.1	0.10	4.46	0.54	19.2	1000	20	139.5	2.23	4.71	2.70	799
40	30	206.9	0.10	6.76	0.54	35.9	1000	30	206.9	2.29	7.13	2.78	1200
40	40	274.1	0.10	8.99	0.54	56.9	1000	40	273.7	2.31	9.72	2.80	1610

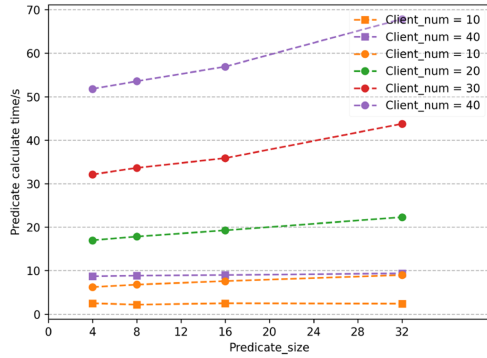


Fig. 6. Runtime of step 2 at *element_num* = 40.

is relatively small, since our scheme requires a longer initialization time for FHE setup and rotation key generation. When the number of elements increases, our scheme demonstrates superior performance. Moreover, Bay's TMPSI runtime grows quadratically with the number of users, while ours increases linearly, showing that our PMPSI protocol scales much better for large participant groups.

C. PMPSI Predicate Size Testing

In our scheme, the *predicate_size* only affects the performance of Step 2. As shown in Fig. 6, bDMOC-DC Protocol is represented by circular markers and DMOC Protocol by square markers for clarity. Although the bDMOC-DC protocol introduces some additional computational overhead, it is negligible compared to the overall computational cost of the algorithm.

By analyzing Fig. 6, changes in *predicate_size* have minimal impact on the runtime of Step 2. Moreover, the experimental results indicate that the runtime of Step 2 increases linearly with *client_num*. This observation suggests that more complex

TABLE VII
COMMUNICATION COST IN PREDICATE COMPUTATION AT *predicate_size* = 16 AND *element_num* = 20

element_num	Setup	DMOC		bDMOC-DC		Step3
		Step 1	Step 2	Step 1	Step 2	
20	14.7MB	8.4MB	42MB	12.6MB	84MB	15.8MB
40	14.7MB	16.8MB	42MB	21MB	84MB	31.6MB
100	14.7MB	84MB	42MB	88MB	84MB	78.9MB
200	14.7MB	168MB	42MB	172MB	84MB	158MB
500	14.7MB	206MB	42MB	211MB	84MB	393MB
1000	14.7MB	407MB	42MB	411MB	84MB	786MB

predicate configurations can be used in practice to achieve finer-grained client selection without incurring significant computational cost.

D. PMPSI Communication Cost Testing

To further evaluate the performance of our implementation, we conducted communication cost testing in Table VII. Since the parameters *predicate_size* and *element_num* have relatively minor influence on the overall communication cost, we set *predicate_size* to 16 and *element_num* to 20 for our experiments.

The use of fully homomorphic encryption in our scheme unavoidably leads to significant communication cost due to the large ciphertext size and the complexity of homomorphic operations. During the Setup phase, the primary operation is the distribution of cryptographic keys, which incurs relatively low communication cost. In Step 1, since we employ a fully homomorphic rotation operation, the user can encrypt multiple bits of the Bloom filter into a single ciphertext. As a result, the communication cost considerably reduced compared with

a bit-wise encryption scheme. While bDMOC-DC incurs additional communication overhead during Step 1 and Step 2, it provides improved privacy, making the overall trade-off acceptable. Meanwhile, Step 3 accounts for the majority of the communication cost, as all users need to collaboratively decrypt the results. Consequently, the communication cost exhibits an approximately linear growth with respect to the number of elements.

Overall, the linear scalability observed in both approaches suggests predictable performance behavior as dataset sizes grow, making the scheme feasible for moderate-scale applications where communication resources are not the primary constraint.

VII. CONCLUSION

This paper introduces a novel primitive, Predicate-Supporting Multi-party Private Set Intersection, which enables a requestor to compute the intersection of datasets with participants whose attributes satisfy a predefined predicate. PMPSI extends traditional MPSI functionality by incorporating fine-grained control and enhanced privacy guarantees, including predicate hiding, attribute hiding, and match status hiding. We construct an efficient protocol combining threshold partially homomorphic encryption, threshold fully homomorphic encryption, and a Dual-Mode Oblivious Comparison sub-protocol, and prove its security under the honest-but-curious model via simulation. Experimental results demonstrate that the PMPSI protocol achieves linear computational and communication complexity with respect to the number of participants, confirming its practicality for real-world applications such as collaborative data sharing and mobile network resource coordination. While efficient under semi-honest assumptions, the current design does not yet address malicious security or further computational optimization. Future work will focus on these directions, as well as broader comparisons with general secure computation frameworks such as MPC circuits to better characterize its efficiency trade-offs.

REFERENCES

- [1] P. Voigt, A. Von, and dem Bussche, *The EU General Data Protection Regulation (gdpr)*. Cham, Switzerland: Springer International Publishing, 2017.
- [2] J. S. Baik, "Data privacy against innovation or against discrimination?: The case of the california consumer privacy act (CCPA)," *Telematics Informat.*, vol. 52, 2020, Art. no. 101431.
- [3] B. Pinkas, M. Rosulek, N. Trieu, and A. Yanai, "PSI from PaXoS: Fast, malicious private set intersection," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2020, pp. 739–767.
- [4] M. Rosulek and N. Trieu, "Compact and malicious private set intersection for small sets," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 1166–1181.
- [5] M. Chase and P. Miao, "Private set intersection in the internet setting from lightweight oblivious PRF," in *Proc. Adv. Cryptology: 40th Annu. Int. Cryptology Conf.*, Santa Barbara, CA, USA, Aug. 2020, pp. 34–63.
- [6] B. Pinkas, T. Schneider, and M. Zohner, "Scalable private set intersection based on ot extension," *ACM Trans. Privacy Secur.*, vol. 21, no. 2, pp. 1–35, 2018.
- [7] M. Ion et al., "On deploying secure computing: Private intersection-sum-with-cardinality," in *Proc. IEEE Eur. Symp. Secur. Privacy*, 2020 pp. 370–389.
- [8] L. Lu and N. Ding, "Multi-party private set intersection in vertical federated learning," in *Proc. IEEE 19th Int. Conf. Trust Secur. Privacy Comput. Commun.*, 2020, pp. 707–714.
- [9] A. Bay, Z. Erkin, J.-H. Hoepman, S. Samardjiska, and J. Vos, "Practical multi-party private set intersection protocols," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 1–15, 2022.
- [10] L. Wei, J. Liu, L. Zhang, and W. Zhang, "Efficient and collusion resistant multi-party private set intersection protocols for large participants and small sets setting," in *Proc. Int. Symp. Cyberspace Saf. Secur.*, 2022, pp. 118–132.
- [11] Y. Gao et al., "Efficient scalable multi-party private set intersection (-variants) from bicentric zero-sharing," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2024, pp. 4137–4151.
- [12] K. Shinagawa, K. Nuida, T. Nishide, G. Hanaoka, and E. Okamoto, "Size-hiding computation for multiple parties," in *Proc. Int. Conf. Theory Application Cryptol. Inf. Secur.*, 2016, pp. 937–966.
- [13] Y. Ishai, J. Kilian, K. Nissim, and E. Petrank, "Extending oblivious transfers efficiently," in *Proc. Annu. Int. Cryptol. Conf.*, 2003, pp. 145–161.
- [14] S. Jarecki and X. Liu, "Efficient oblivious pseudorandom function with applications to adaptive ot and secure computation of set intersection," in *Proc. Theory Cryptogr., 6th Theory Cryptogr. Conf.*, San Francisco, CA, USA, Mar. 2009, pp. 577–594.
- [15] V. Kolesnikov, N. Matania, B. Pinkas, M. Rosulek, and N. Trieu, "Practical multi-party private set intersection from symmetric-key techniques," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1257–1272.
- [16] F. Karakoç and A. Küpçü, "Linear complexity private set intersection for secure two-party protocols," in *Proc. Int. Conf. Cryptol. Netw. Secur.*, 2020, pp. 409–429.
- [17] G. Garimella, B. Pinkas, M. Rosulek, N. Trieu, and A. Yanai, "Oblivious key-value stores and amplification for private set intersection," in *Proc. Adv. in Cryptology- 41st Annu. Int. Cryptology Conf.*, 2021, pp. 395–425.
- [18] S. Raghuraman and P. Rindal, "Blazing fast PSI from improved OKVS and subfield VOLE," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 2505–2517.
- [19] D. Morales, I. Agudo, and J. Lopez, "Private set intersection: A systematic literature review," *Comput. Sci. Rev.*, vol. 49, 2023, Art. no. 100567.
- [20] A. Davidson and C. Cid, "An efficient toolkit for computing private set operations," in *Inf. Secur. Privacy: 22nd Australas. Conf.*, Auckland, New Zealand, Jul. 2017, pp. 261–278.
- [21] H. Chen, K. Laine, and P. Rindal, "Fast private set intersection from homomorphic encryption," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1243–1255.
- [22] O. Nevo, N. Trieu, and A. Yanai, "Simple, fast malicious multiparty private set intersection," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 1151–1165.
- [23] S. Zhao et al., "Multi-party private set intersection with one-round online interaction," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 5549–5562, 2025.
- [24] G. Ateniese, E. De Cristofaro, and G. Tsudik, "(If) Size matters: Size-hiding private set intersection," in *Proc. 14th Int. Conf. Pract. Theory Public Key Cryptography*, Taormina, Italy, Mar. 2011, pp. 156–173.
- [25] M. Blanton and E. Aguiar, "Private and oblivious set and multiset operations," in *Proc. 7th ACM Symp. Inf.*, 2012, pp. 40–41.
- [26] P. Rindal and P. Schoppmann, "VOLE-PSI: Fast OPRF and circuit-PSI from vector-OLE," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2021, pp. 901–930.
- [27] Y. Yang, J. Weng, Y. Yi, C. Dong, L. Y. Zhang, and J. Zhou, "Predicate private set intersection with linear complexity," in *Proc. Int. Conf. Appl. Cryptogr. Netw. Secur.*, 2023, pp. 143–166.
- [28] G. Ling et al., "P² FRPSI: Privacy-preserving feature retrieved private set intersection," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 2201–2216, 2024.
- [29] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Commun. ACM*, vol. 13, no. 7, pp. 422–426, 1970.
- [30] P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes," in *Proc. Int. Conf. Theory Appl. Cryptographic Techn.*, 1999, pp. 223–238.
- [31] A. Lysyanskaya and C. Peikert, "Adaptive security in the threshold setting: From cryptosystems to signature schemes," in *Proc. Adv. Crypt. Int. Conf. Theory Appl. Crypt. Info. Sec.*, 2001, pp. 331–350.
- [32] D. Boneh et al., "Threshold cryptosystems from threshold fully homomorphic encryption," in *Proc. Adv. Cryptology 38th Annu. Int. Crypt. Conf.*, Santa Barbara, CA, USA, 2018, pp. 565–596.
- [33] J. Katz, A. Sahai, and B. Waters, "Predicate encryption supporting disjunctions, polynomial equations, inner products," in *Proc. Adv. Cryptology 27th Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, Istanbul, Türkiye, 2008, pp. 146–162.

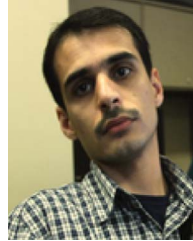
- [34] Y. Zheng et al., "SetRkNN: Efficient and privacy-preserving set reverse kNN query in cloud," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 888–903, 2023.
- [35] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, "Leveled fully homomorphic encryption without bootstrapping," *ACM Trans. Comput. Theory*, vol. 6, no. 3, pp. 1–36, 2014.



Hongyi Liu received the PhD degree from Peking University in 2025. He is currently an assistant research professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology, China. He published more than 15 papers in international journals and conferences. His research interests include cloud computing, system reliability, anomaly detection, and artificial intelligence security.



Chengzhi Gao received the BS degree in 2024 from School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing, China, where he is currently working toward the master's degree. His research interests include Multi-party privacy computing and oblivious transfer.



Kashif Sharif (Senior Member, IEEE) received the MS degree in information technology in 2004, and PhD degree in computing and informatics from University of North Carolina, Charlotte, USA in 2012. He is currently an associate professor with Beijing Institute of Technology, China. His research interests include network intelligence, mobile computing, B5G communications, and network & communications security. He is an associate editor of IEEE Journals.



Yihan Bao received the BS degree in 2024 from the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing, China, where he is currently working toward the PhD degree. His research interests include privacy computing, attribute-based encryption, and lattice-based cryptography.



Pengbo Wang received the MS degree from the School of Control and Computer Engineering, North China Electric Power University, China, in 2021. He is currently working toward the PhD degree with the School of Cyberspace Science and Technology, Beijing Institute of Technology, China. His research interests include privacy computing and attribute-based encryption.



Chang Xu received the PhD degree in computer science from Beihang University, Beijing, China, in 2013. She is currently an associate professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology. Her research interests include security & privacy in VANET and Big Data security.



Liehuang Zhu (Senior Member, IEEE) received the PhD degree in computer science from Beijing Institute of Technology, Beijing, China, in 2004. He is currently a professor with the School of Cyberspace Science and Technology, Beijing Institute of Technology. His research interests include security protocol analysis and design, group key exchange protocols, wireless sensor networks, and cloud computing.